

How to transfer data from controllers

Objectives (part 1)

Applied

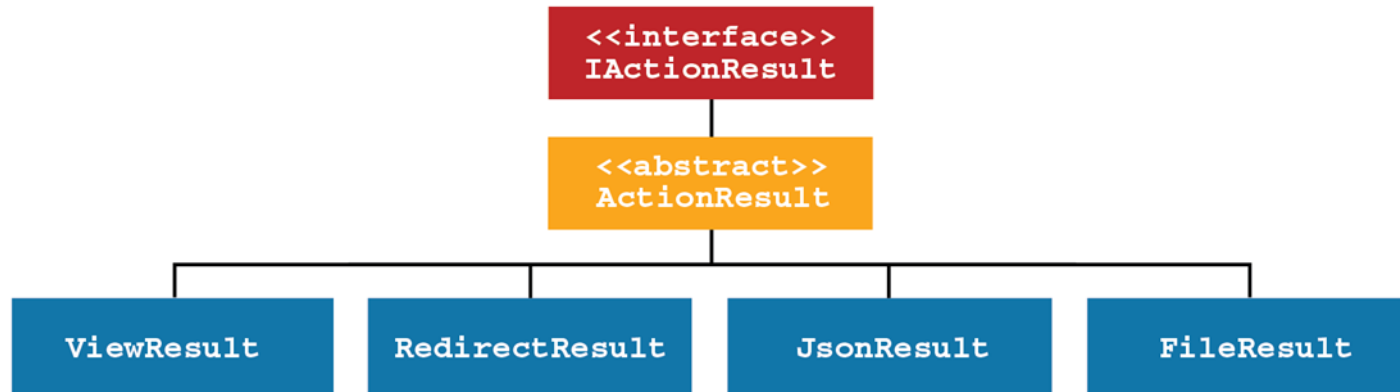
1. Use the ViewBag and ViewData properties to transfer data from a controller to a view.
2. Use a view model to transfer data from a controller to a strongly-typed view.
3. Develop web apps that redirect and use the TempData property to transfer data from a controller to a controller.

Objectives (part 2)

Knowledge

1. Describe some important interfaces and classes of the ActionResult hierarchy.
2. Distinguish between the ViewBag and ViewData properties.
3. Describe the use of a view model to transfer data from a controller to a view.
4. Describe how one action method can redirect to another action method.
5. Describe the use of the PRG pattern to prevent resubmission of POST data.
6. Distinguish between the ViewData and TempData properties.
7. Describe the purpose of the Keep() and Peek() methods of the TempData class.

The ActionResult hierarchy



Common ActionResult subtypes

`ViewResult`

`RedirectResult`

`RedirectToActionResult`

`JsonResult`

`FileResult`

`StatusCodeResult`

`ContentResult`

`EmptyResult`

A URL for a full list of the ActionResult subtypes

<https://docs.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.mvc.actionresult>

Some methods of the Controller class that return an ActionResult object

```
View()  
Redirect()  
RedirectToAction()  
File()  
Json()
```

Some of the overloads of the View() method

```
View()  
View(model)  
View(name)  
View(name, model)
```

An action method that returns a ViewResult object

```
public ViewResult List() {  
    var names = new List<string> { "Grace", "Ada", "Charles" };  
    return View(names);  
}
```

An action method that returns a RedirectToActionResult object

```
public RedirectToActionResult Index() =>  
    RedirectToAction("List");
```

An action method that may return different types of result objects

```
[HttpPost]  
public IActionResult Edit(string id) {  
    if (ModelState.IsValid)  
        return RedirectToAction("List");  
    else  
        return View((object)id);    // cast string model to object  
}
```

Controller code that adds two items to the ViewData property

```
public IActionResult Index() {  
    ViewData["Book"] = "Alice in Wonderland";  
    ViewData["Price"] = 9.99;  
    return View();  
}
```

Some properties of the ViewDataDictionary class

Count

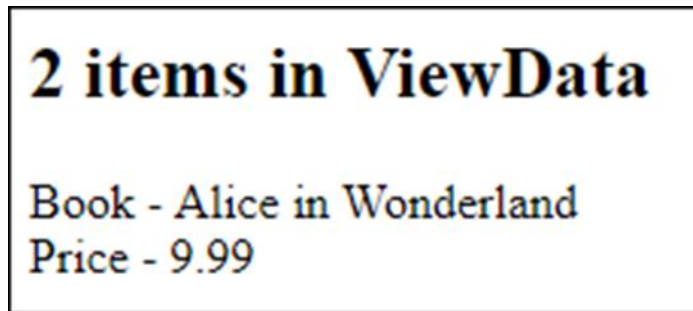
Keys

Values

Razor code that displays all the items in the ViewData object

```
<h2>@ViewData.Count items in ViewData</h2>
@foreach (KeyValuePair<string, object> item in ViewData)
{
    <div>@item.Key - @item.Value</div>
}
```

The view in the browser



Razor code that casts a ViewData object to the double type

```
<h4>Price:  
@(((double)ViewData["Price"]).ToString("c"))</h4>
```

The view in the browser

Price: \$9.99

Razor code that checks ViewData values for null

Non-nullable type (double)

```
<h4>Price: @ ( (ViewData["Price"] == null) ? "" :  
                ((double)ViewData["Price"]).ToString("c"))</h4>
```

Nullable type (string)

```
<h4>Book: @ViewData["Book"]?.ToString()</h4>
```

Controller code that adds two dynamic properties to the ViewBag property

```
public IActionResult Index() {  
    ViewBag.Book = "Alice in Wonderland";  
    ViewBag.Price = 9.99;  
    return View();  
}
```

Razor code that uses ViewData to display ViewBag properties

```
<h2>@ViewData.Count ViewBag properties</h2>
@foreach (KeyValuePair<string, object> item
in ViewData) {
    <div>@item.Key - @item.Value</div>
}
```

The view in the browser

2 ViewBag properties

Book - Alice in Wonderland
Price - 9.99

Razor code that works with a ViewBag property without casting

```
<h4>Price: @ViewBag.Price.ToString("c")</h4>
```

The view in the browser

Price: \$9.99

Razor code that checks ViewBag properties for null

Non-nullable type (double)

```
<h4>Price: @ViewBag.Price?.ToString("c")</h4>
```

Nullable type (string)

```
<h4>Book: @ViewBag.Book?.ToString()</h4>
```

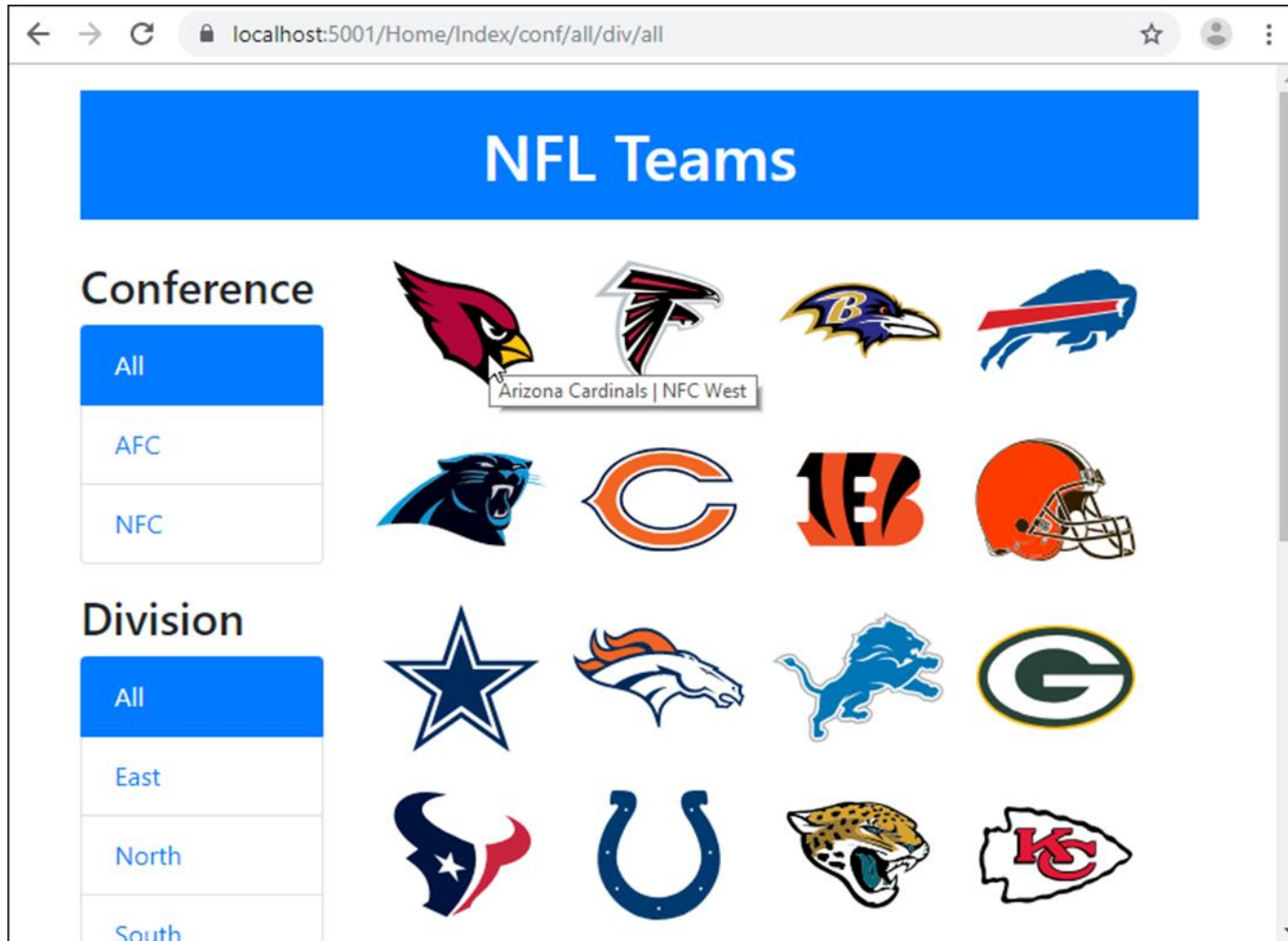
Use ViewData instead of ViewBag when you need to...

- Use a key name that isn't valid in C#, such as a key that contains spaces.
- Call properties and methods of the ViewDataDictionary class, such as its Count property or its Clear() method.
- Loop through all items in the ViewData dictionary.

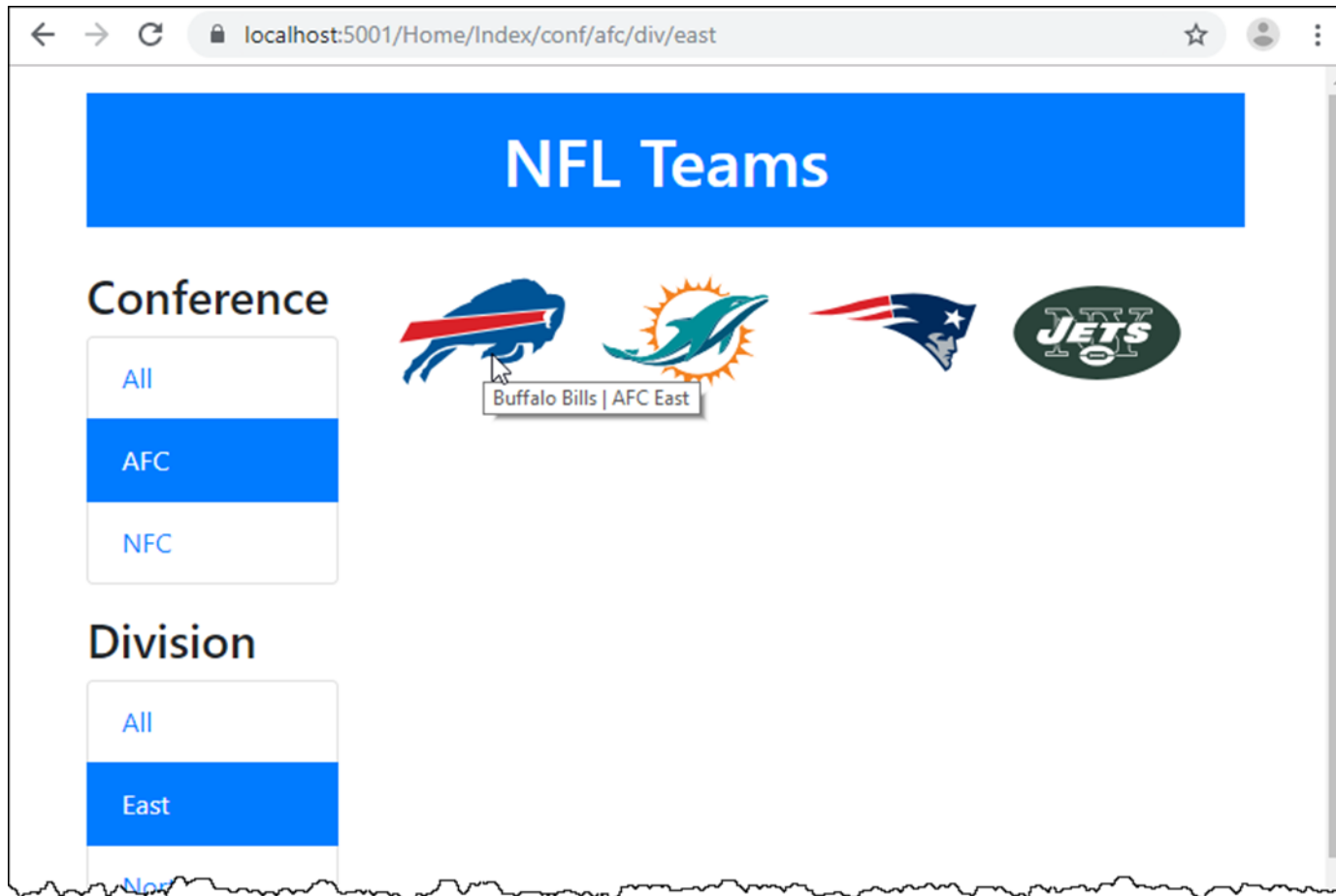
How the teams of the NFL are organized

- There are two conferences, the NFC and the AFC.
- Each conference contains four divisions named North, South, East, and West.
- Each division contains four teams.

The NFL Teams app on first load



The NFL Teams app after a conference and division are selected



The Conference class

```
public class Conference
{
    public string ConferenceID { get; set; }
    public string Name { get; set; }
}
```

The Division class

```
public class Division
{
    public string DivisionID { get; set; }
    public string Name { get; set; }
}
```

The Team class

```
public class Team
{
    public string TeamID { get; set; }
    public string Name { get; set; }
    public Conference Conference { get; set; }
    public Division Division { get; set; }
    public string LogoImage { get; set; }
}
```

The TeamContext class (part 1)

```
public class TeamContext : DbContext
{
    public TeamContext(DbContextOptions<TeamContext> options)
        : base(options) { }

    public DbSet<Team> Teams { get; set; }
    public DbSet<Conference> Conferences { get; set; }
    public DbSet<Division> Divisions { get; set; }

    protected override void OnModelCreating(
        modelBuilder) {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Conference>().HasData(
            new Conference { ConferenceID = "afc", Name = "AFC"},
            ...
        );

        modelBuilder.Entity<Division>().HasData(
            new Division { DivisionID = "north", Name = "North" },
            ...
        );
    }
}
```

The TeamContext class (part 2)

```
        modelBuilder.Entity<Team>().HasData(  
            new {TeamID = "ari", Name = "Arizona Cardinals",  
                ConferenceID = "nfc", DivisionID = "west",  
                LogoImage = "ARI.png"},  
            ...  
        );  
    }  
}
```

The Home controller (part 1)

```
public class HomeController : Controller
{
    private TeamContext context;

    public HomeController(TeamContext ctx)
    {
        context = ctx;
    }

    public ViewResult Index(string activeConf = "all",
                           string activeDiv = "all")
    {
        // store selected conference and division IDs in view bag
        ViewBag.ActiveConf = activeConf;
        ViewBag.ActiveDiv = activeDiv;
    }
}
```

The Home controller (part 2)

```
// get list of conferences and divisions from database
List<Conference> conferences = context.Conferences.ToList();
List<Division> divisions = context.Divisions.ToList();

// insert default "All" value at front of each list
conferences.Insert(0, new Conference { ConferenceID = "all",
                                     Name = "All" });
divisions.Insert(0, new Division { DivisionID = "all",
                                  Name = "All" });

// store lists in view bag
ViewBag.Conferences = conferences;
ViewBag.Divisions = divisions;
```

The Home controller (part 3)

```
// get teams - filter by conference and division
IQueryable<Team> query = context.Teams;
if (activeConf != "all")
    query = query.Where(
        t => t.Conference.ConferenceID.ToLower() ==
            activeConf.ToLower());
if (activeDiv != "all")
    query = query.Where(
        t => t.Division.DivisionID.ToLower() ==
            activeDiv.ToLower());

// pass teams to view as model
var teams = query.ToList();
return View(teams);
}
}
```

The custom route in the Startup.cs file

```
endpoints.MapControllerRoute(  
    name: "custom",  
    pattern:  
        "{controller}/{action}/conf/{activeConf}/div/{activeDiv}");  
  
endpoints.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

The layout

```
<!DOCTYPE html>

<html>
<head>
  <meta name="viewport" content="width=device-width" />
  <link href="/lib/bootstrap/dist/css/bootstrap.min.css"
        rel="stylesheet" />
  <link href="/css/site.css" rel="stylesheet" />
  <title>@ViewBag.Title</title>
</head>
<body>
  <div class="container">
    <header class="text-center text-white">
      <h1 class="bg-primary mt-3 p-3">NFL Teams</h1>
    </header>
    <main>
      @RenderBody()
    </main>
  </div>
</body>
</html>
```

The Home/Index view (NFL Teams 1.0) (part 1)

```
@model IEnumerable<Team>
@{
    ViewData["Title"] = "NFL Teams";
    string Active(string filter, string selected)
    {
        return (filter.ToLower() == selected.ToLower()) ?
            "active" : "";
    }
}
<div class="row">
    <div class="col-sm-3">
        <h3 class="mt-3">Conference</h3>
        <div class="list-group">
            @foreach (Conference conf in ViewBag.Conferences)
            {
                <a asp-action="Index"
                    asp-route-activeConf="@conf.ConferenceID"
                    asp-route-activeDiv="@ViewBag.ActiveDiv"
                    class="list-group-item @Active(conf.ConferenceID,
                                                    ViewBag.ActiveConf)">

                    @conf.Name
                </a>
            }
        </div>
    </div>
```

The Home/Index view (NFL Teams 1.0) (part 2)

```
<h3 class="mt-3">Division</h3>
<div class="list-group">
  @foreach (Division div in ViewBag.Divisions)
  {
    <a asp-action="Index"
      asp-route-activeConf="@ViewBag.ActiveConf"
      asp-route-activeDiv="@div.DivisionID"
      class="list-group-item @Active(div.DivisionID,
                                   ViewBag.ActiveDiv) ">
      @div.Name
    </a>
  }
</div>
</div>
```

The Home/Index view (NFL Teams 1.0) (part 3)

```
<div class="col-sm-9">
  <ul class="list-inline">
    @foreach (Team team in Model)
    {
      <li class="list-inline-item">
        
      </li>
    }
  </ul>
</div>
</div>
```

A view model for the NFL Teams app (part 1)

```
public class TeamListViewModel
{
    public List<Team> Teams { get; set; }
    public string ActiveConf { get; set; }
    public string ActiveDiv { get; set; }

    // make next two properties standard properties so the setter
    // can make the first item in each list "All"
    private List<Conference> conferences;
    public List<Conference> Conferences {
        get => conferences;
        set {
            conferences = value;
            conferences.Insert(0,
                new Conference { ConferenceID = "all",
                               Name = "All" });
        }
    }
}
```

A view model for the NFL Teams app (part 2)

```
private List<Division> divisions;
public List<Division> Divisions {
    get => divisions;
    set {
        divisions = value;
        divisions.Insert(0,
            new Division { DivisionID = "all",
                           Name = "All" });
    }
}

// methods to help view determine active link
public string CheckActiveConf(string c) =>
    c.ToLower() == ActiveConf.ToLower() ? "active" : "";

public string CheckActiveDiv(string d) =>
    d.ToLower() == ActiveDiv.ToLower() ? "active" : "";
}
```

The updated Index() action method of the Home controller

```
public IActionResult Index(string activeConf = "all",
    string activeDiv = "all")
{
    var model = new TeamListViewModel {
        ActiveConf = activeConf,
        ActiveDiv = activeDiv,
        Conferences = context.Conferences.ToList(),
        Divisions = context.Divisions.ToList()
    };
    IQueryable<Team> query = context.Teams;
    if (activeConf != "all")
        query = query.Where(t =>
            t.Conference.ConferenceID.ToLower() ==
                activeConf.ToLower());
    if (activeDiv != "all")
        query = query.Where(t =>
            t.Division.DivisionID.ToLower() ==
                activeDiv.ToLower());
    model.Teams = query.ToList();
    return View(model);
}
```

The updated Home/Index view (part 1)

```
@model TeamListViewModel
@{
    ViewData["Title"] = "NFL Teams"; // helper function not needed
}
<div class="row">
    <div class="col-sm-3">
        <h3>Conference</h3>
        <div class="list-group">
            @foreach (Conference conf in Model.Conferences) {
                <a asp-action="Index"
                    asp-route-conference="@conf.ConferenceID"
                    asp-route-division="@Model.ActiveDiv"
                    class="list-group-item"
                    @Model.CheckActiveConf(conf.ConferenceID)">
                    @conf.Name</a>
            }
        </div>
    </div>
```

The updated Home/Index view (part 2)

```
<h3>Division</h3>
<div class="list-group">
    @foreach (Division div in Model.Divisions) {
        <a asp-action="Index"
            asp-route-conference="@Model.ActiveConf"
            asp-route-division="@div.DivisionID"
            class="list-group-item"
            @Model.CheckActiveDiv(div.DivisionID) ">
            @div.Name</a>
    }
</div>
</div>
<div class="col-sm-9">
    <ul class="list-inline">
        @foreach (Team team in Model.Teams) {
            <!-- same as figure 8-9 -->
        }
    </ul>
</div>
</div>
```

Two of the HTTP status codes for redirection

302 Found

301 Moved Permanently

The ActionResult subtypes for redirection

Subtype	302 Found method	301 Moved Permanently method
<code>RedirectResult</code>	<code>Redirect()</code>	<code>RedirectPermanent()</code>
<code>LocalRedirectResult</code>	<code>LocalRedirect()</code>	<code>LocalRedirectPermanent()</code>
<code>RedirectToActionResult</code>	<code>RedirectToAction()</code>	<code>RedirectToActionPermanent()</code>
<code>RedirectToRouteResult</code>	<code>RedirectToRoute()</code>	<code>RedirectToRoutePermanent()</code>

How to know which subtype to use for redirection

Subtype	Use when...
<code>RedirectResult</code>	Redirecting to an external URL, such as https://google.com .
<code>LocalRedirectResult</code>	Making sure you redirect to a URL within the current app.
<code>RedirectToActionResult</code>	Redirecting to an action method within the current app.
<code>RedirectToRouteResult</code>	Redirecting within the current app by using a named route.

Some of the overloads available for the RedirectToAction() method

Arguments	Redirect to...
<code>(a)</code>	The specified action method in the current controller.
<code>(a, c)</code>	The specified action method in the specified controller.
<code>(a, routes)</code>	The specified action method in the current controller with route parameters.
<code>(a, c, routes)</code>	The specified action method in the specified controller with route parameters.

Code that redirects to another action method (part 1)

The List() action method in the current controller

```
public RedirectToActionResult Index() =>  
    RedirectToAction("List");
```

The List() action method in the Team controller

```
public RedirectToActionResult Index() =>  
    RedirectToAction("List", "Team");
```

The Details() action method in the current controller with a parameter

```
public RedirectToActionResult Index(string id) =>  
    RedirectToAction("Details", new { ID = id });
```

Code that redirects to another action method (part 2)

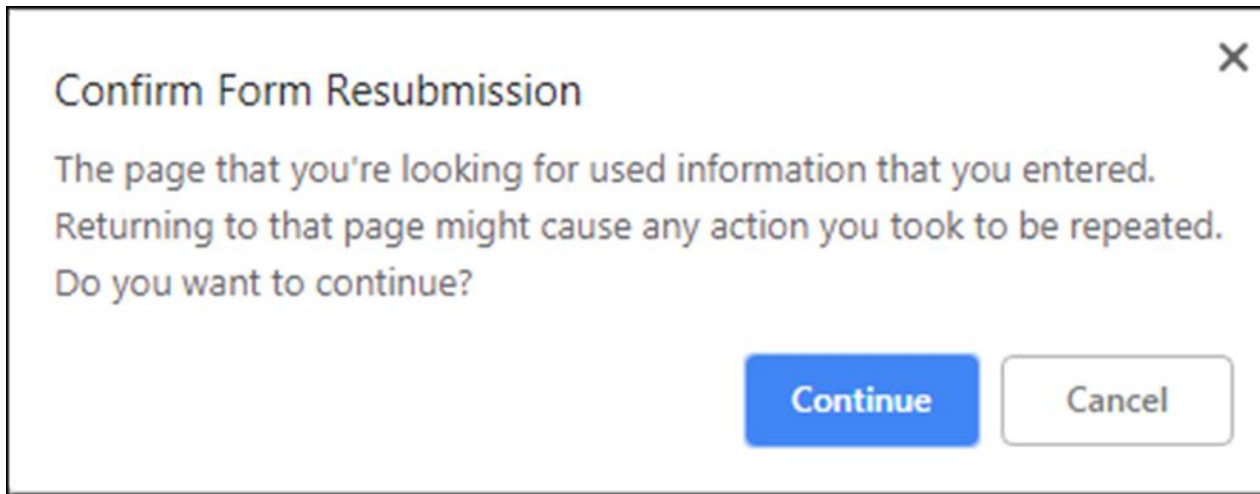
Use a shortcut when variable name and route segment name match

```
public RedirectToActionResult Index(string id) =>
    RedirectToAction("Details", new { id });
```

Use a string-string dictionary to supply a parameter

```
public RedirectToActionResult Index(string id) =>
    RedirectToAction("Details",
        new Dictionary<string, string>() { { "ID", id } }
    );
```

A browser message that's displayed when you refresh a page displayed by a POST request



Action methods that use the PRG pattern (part 1)

A Delete() action method for a POST request

[HttpPost]

```
// Post
public IActionResult Delete(Movie movie)
{
    context.Movies.Remove(movie);
    context.SaveChanges();
    return RedirectToAction("Index", "Home");
// Redirect
}
```

Action methods that use the PRG pattern (part 2)

The Index() action method of the Home controller
for a GET request

[HttpGet]

```
// Get
public IActionResult Index()
{
    var movies = context.Movies
        .Include(m => m.Genre)
        .OrderBy(m => m.Name)
        .ToList();
    return View(movies);
}
```

An action method that uses TempData with the PRG pattern

```
[HttpPost]
public IActionResult Delete(Movie movie)
{
    context.Movies.Remove(movie);
    context.SaveChanges();
    TempData["message"] =
        $"{movie.Name} deleted from database.";
    return RedirectToAction("Index", "Home");
}
```

Code that reads a TempData value in a Layout view

```
...  
<header class="jumbotron">  
    <h1>My Movies</h1>  
</header>...  
@if (TempData.Keys.Contains("message"))  
{  
    <h4 class="bg-info text-center text-white p-2">  
        @TempData["message"]  
    </h4>  
}  
...
```

The temporary message displayed in a browser



Two methods of the TempDataDictionary class

Keep ()

Keep (key)

Peek (key)

The overloaded Details() action method of the Home controller (part 1)

[HttpPost]

```
public RedirectToActionResult Details(TeamViewModel model)
{
    Utility.LogTeamClick(model.Team.TeamID);
    TempData["ActiveConf"] = model.ActiveConf;
    TempData["ActiveDiv"] = model.ActiveDiv;
    return RedirectToAction("Details",
                           new { ID = model.Team.TeamID });
}
```

The overloaded Details() action method of the Home controller (part 2)

[HttpGet]

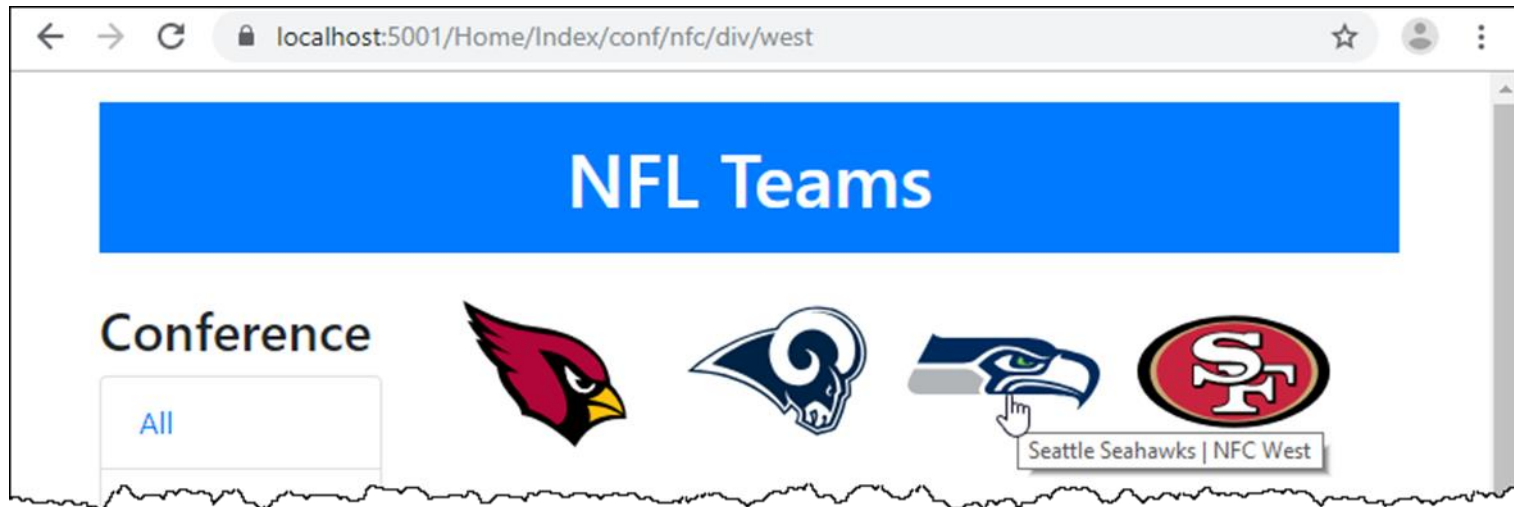
```
public IActionResult Details(string id)
{
    Utility.LogTeamClick(model.Team.TeamID);

    var model = new TeamViewModel {
        Team = context.Teams
            .Include(t => t.Conference)
            .Include(t => t.Division)
            .FirstOrDefault(t => t.TeamID == id),
        ActiveConf = TempData.Peek("ActiveConf").ToString(),
        ActiveDiv = TempData.Peek("ActiveDiv").ToString()
    };
    return View(model);
}
```

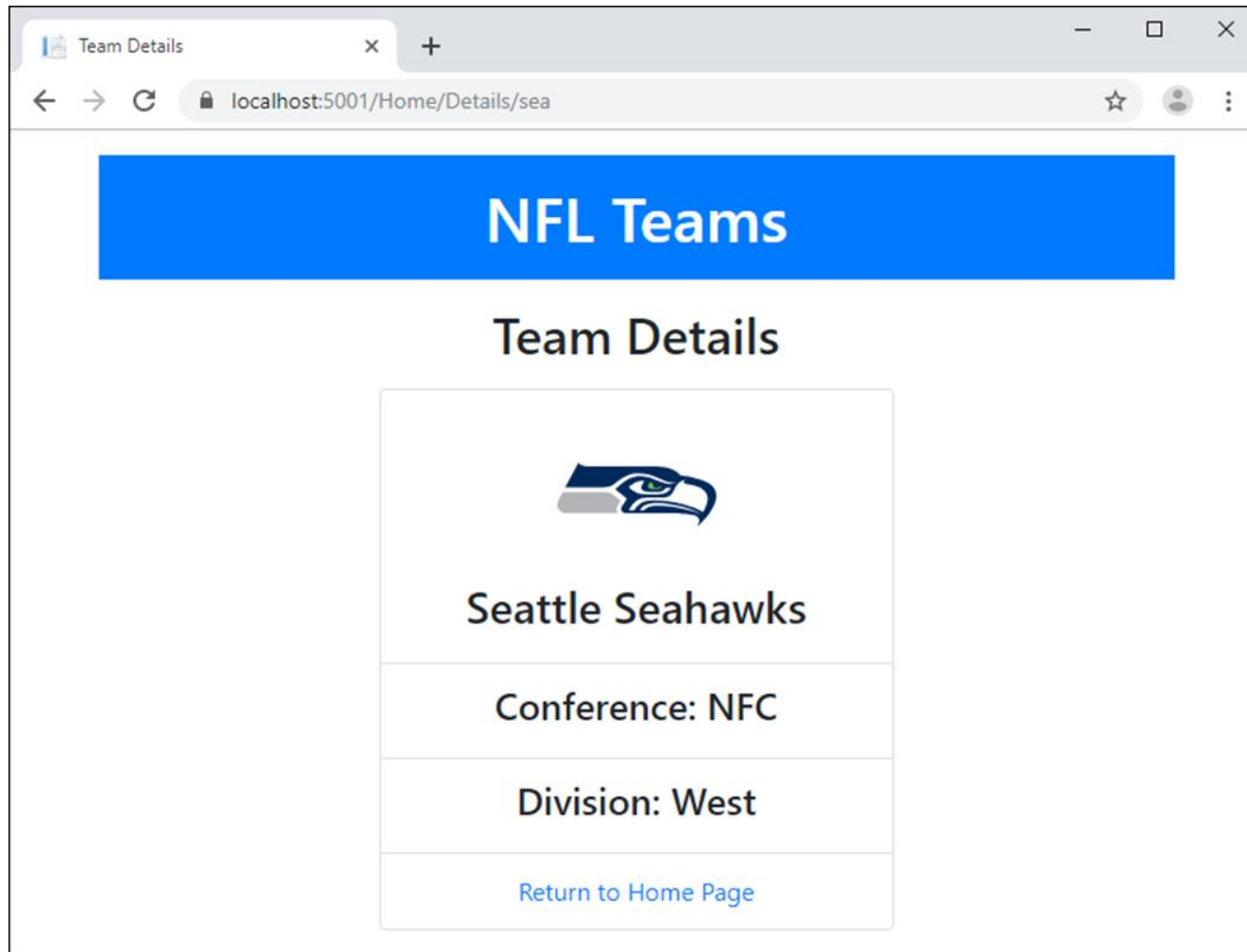
When to use the `Keep()` and `Peek()` methods

- Use `Peek()` when you know you want the value to stay marked as unread.
- Use a normal read and `Keep()` when you want to use a condition to determine whether to mark the value as unread.

The Home page after selecting a conference and division



The Details page after clicking on a team



The Home page after clicking on the “Return to Home Page” link



The TeamViewModel class

```
public class TeamViewModel
{
    public Team Team { get; set; }
    public string ActiveConf { get; set; } = "all";
    public string ActiveDiv { get; set; } = "all";
}
```

The updated TeamListViewModel class

```
public class TeamListViewModel : TeamViewModel
{
    // Team, ActiveConf, and ActiveDiv properties are inherited
    // Teams, Conferences, and Divisions properties same as
    // figure 8-10
    // CheckActiveConf() and CheckActiveDiv() methods same as
    // figure 8-10
}
```

The Details() action method of the Home controller (part 1)

```
[HttpPost]
public RedirectToActionResult Details(TeamViewModel model)
{
    Utility.LogTeamClick(model.Team.TeamID);

    TempData["ActiveConf"] = model.ActiveConf;
    TempData["ActiveDiv"] = model.ActiveDiv;
    return RedirectToAction("Details",
        new { ID = model.Team.TeamID });
}
```

The Details() action method of the Home controller (part 2)

```
[HttpGet]
public IActionResult Details(string id)
{
    var model = new TeamViewModel
    {
        Team = context.Teams
            .Include(t => t.Conference)
            .Include(t => t.Division)
            .FirstOrDefault(t => t.TeamID == id),
        ActiveDiv = TempData?["ActiveDiv"]?.ToString() ?? "all",
        ActiveConf = TempData?["ActiveConf"]?.ToString() ?? "all"
    };
    return View(model);
}
```

The Home/Index view (NFL Teams 2.0) (part 1)

```
@model TeamListViewModel
```

```
@{
```

```
    ViewData["Title"] = "NFL Teams";
```

```
}
```

```
<div class="row">
```

```
    <div class="col-sm-3">
```

```
        <!-- Conference and Division ul elements -->
```

```
    </div>
```

```
    <div class="col-sm-9">
```

```
        <ul class="list-inline">
```

```
            @foreach (Team team in Model.Teams)
```

```
            {
```

```
                <li class="list-inline-item">
```

```
                    <form asp-action="Details" method="post">
```

```
                        <button class="bg-white border-0"
```

```
                            type="submit">
```

```
                            
```

```
                    </button>
```

The Home/Index view (NFL Teams 2.0) (part 2)

```
        <input type="hidden" asp-for="@team.TeamID" />
        <input type="hidden" asp-for="ActiveConf" />
        <input type="hidden" asp-for="ActiveDiv" />
    </form>
</li>
}
</ul>
</div>
</div>
```

The Home/Details view (part 1)

```
@model TeamViewModel
```

```
@{  
    ViewData["Title"] = "Team Details";  
}
```

```
<h2>Team Details</h2>
```

```
<div class="row">  
    <div class="col-sm-6 col-sm-offset-3">  
        <ul class="list-group text-center">  
            <li class="list-group-item">  
                  
                <h3>@Model.Team.Name</h3>  
            </li>  
            <li class="list-group-item">  
                <h4>Conference: @Model.Team.Conference.Name</h4>  
            </li>  
            <li class="list-group-item">  
                <h4>Division: @Model.Team.Division.Name</h4>  
            </li>  
        </ul>  
    </div>  
</div>
```

The Home/Details view (part 2)

```
<li class="list-group-item">
  <a asp-action="Index"
    asp-route-conference="@Model.ActiveConf"
    asp-route-division="@Model.ActiveDiv">
    Return to Home Page
  </a>
</li>
</ul>
</div>
</div>
```